Conditional Timed Partial Orders: An Expressive and Interpretable Framework for Robot Task Specification and Planning

Sebastian Escobar, Morteza Lahijanian

***Abstract*— Robot tasks are becoming increasingly complex, requiring specification languages that are both expressive and human-interpretable while enabling tractable planning. Timed Partial Orders (TPOs), originally proposed for workflows, provide an interpretable specification framework with planning algorithms based on mixed-integer linear programming (MILP). However, TPOs are limited in expressivity, capturing only partial-order events with simple timing constraints. In this paper, we introduce Conditional TPOs (cTPOs), which extend TPOs with richer relative-timing constraints and conditional event activations based on environmental conditions. We show that planning for cTPOs also reduces to an MILP problem; however, the added expressivity results in significantly larger MILPs that can become computationally intractable. To address this challenge, we propose a decomposition algorithm that partitions a cTPO into smaller sub-TPOs, yielding a sequence of smaller MILP problems. We prove that this decomposition is complete and preserves plan optimality while improving the interpretability of complex tasks. Experimental results demonstrate the effectiveness of cTPOs as a task specification framework and the efficiency of our decomposition approach, achieving up to three orders of magnitude speedup over a monolithic MILP formulation.**

I. Introduction

Robotic autonomy increasingly requires reasoning at the mission level: robots must decide not only *how* to move, but *what* to do, *when* to do it, and under which conditions tasks become required. Missions in domains such as healthcare, aerospace, and field robotics are structured by precedence, deadlines, and conditional obligations. Representations that capture these requirements while remaining interpretable and computationally tractable are an active area of research [1].

Timed Partial Orders (TPOs) [2] model structured missions as directed acyclic graphs (DAGs) whose edges encode task precedence and whose clocks encode local timing windows. This representation lets designers to read, modify, and verify requirements directly from the graph while reducing timing constraints to linear difference constraints, avoiding the clock-region blowup of Timed Automata [3], [4]. Planning under a TPO specification can be formulated as a Generalized Traveling Salesman Problem with Time Windows and Precedence Relations (GTSP-TWPR) [5]–[7] and solved via Mixed-Integer Linear Programming (MILP) [8]–[10], which, though NP-hard [11], is tractable for practical problem sizes.

Temporal logics that extend Linear Temporal Logic with explicit timing bounds, such as Metric Temporal Logic (MTL) [12], have also been applied to robotic task specification and planning. By augmenting temporal operators with quantitative time intervals, MTL can express rich timing constraints and has been used in planning and control synthesis with MILP formulations [13], [14]. While expressive, MTL specifications are abstract formulas over execution traces, making it difficult to directly interpret or modify timing relationships between individual tasks. In contrast, TPOs encode timing constraints as annotated precedence graphs, providing a structured and visualizable representation that is easier for designers to inspect, validate, and adjust.

Despite their strengths, existing TPOs have two expressiveness limitations. First, guard conditions are restricted to single clocks reset by a unique predecessor, so timing constraints can only relate precedence-ordered events. Constraints between concurrent events without a direct ordering cannot be expressed. Simple Temporal Networks (STNs) [15] support arbitrary pairwise time-difference constraints but provide no model of a planning environment or task locations, making them unsuitable as a direct replacement. Second, the task structure is fixed at specification time: subtasks and their timing constraints cannot be conditionally activated based on the robot's realized trajectory.

Conditional task activation has been addressed primarily through reactive synthesis. H. Kress-Gazit et al. [16], [17] synthesize controllers that branch behavior based on environmental observations at runtime, and M. Lahijanian et al. [18] demonstrate deployment in environments where plans must adapt to encountered conditions. In each case, specifications are LTL formulas over Boolean propositions: a task is required when a condition is observed, but no mechanism enforces quantitative timing constraints between conditionally activated tasks and the rest of the plan.

We address both gaps with *conditional Timed Partial Orders* (cTPOs), introduced in this paper. cTPOs extend TPOs with generalized clock-difference guards that relate any pair of events and propositional activation conditions that conditionally enforce tasks and their timing constraints based on the realized trajectory. We further extend the MILP formulation of [8] to handle cTPO specifications and introduce *sub-TPO decomposition*, which identifies self-contained subgraphs that can be solved independently. Decomposition yields both a substantial reduction in computation time and an interpretability gain by exposing the modular structure of the mission specification.

II. TPOs & Planning Problem

We consider complex robotic tasks defined over a set of events with temporal requirements. A formalism that provides easily interpretable visual representations of such

Authors are with the Aerospace Eng. Sci. Dept. at University of Colorado Boulder, CO, USA {firstname.lastname}@colorado.edu

tasks is Timed Partial Orders (TPOs). In this section, we first formally define TPOs and discuss their limitations in expressing complex tasks. We then introduce Conditional TPOs (cTPOs) to address these limitations and finally formulate the cTPO planning problem.

A. Timed Partial Order

TPOs use partial-order relations together with clocks to capture temporal requirements over events, resulting in an annotated Directed Acyclic Graph (DAG) representation.

Definition 1 (TPO [2]). *A* Timed Partial Order *(TPO) is a tuple* $\phi = (\Pi, \prec, C, \mathcal{G}_C, G, R)$*, where*

- $\Pi = \{e_1, \ldots, e_n\}$ *is a finite set of events;*
- $\prec \subseteq \Pi \times \Pi$ *is a strict partial order over events in* Π*;*
- $C = \{c_1, \ldots, c_m\}$ *is a finite set of clocks;*
- $\mathcal{G}_C = \{ \bigwedge_{i \in I}(c_i \bowtie t_i) \mid I \subseteq \{1, \ldots, |C|\}, \bowtie \in \{<, \leq, \geq, >\}, t_i \in \mathbb{R}_{\geq 0} \}$ *is the set of timing constraints (guard conditions), each in the form of conjunctions of clock bounds;*
- $G : \Pi \to \mathcal{G}_C$ *assigns to an event* $e \in \Pi$ *a timing constraint (guard condition)* $G(e) \in \mathcal{G}_C$*; and*
- $R : \Pi \to 2^C$ *is the reset map that, uppon execution of an event* e*, resets clocks* $R(e) \subseteq C$*.*

A TPO can be represented as a DAG whose edges encode the cover relation of $\prec$. We illustrate a TPO specification in the example below.

Example 1 (Hospital Robot Workflow). *Consider a hospital robot tasked with collecting diagnostic results and delivering them to medical staff. The task is: first collect the triage report* (e_1)*; then perform the blood draw* (e_2) *and imaging scan* (e_3) *in any order; finally deliver the results to a physician* (e_4) *and the patient file to a social worker* (e_5) *in any order. The blood draw* e_2 *and physician delivery* e_4 *must occur within 15 and 60 minutes, respectively, of triage collection* e_1*.*

The precedence relations are $e_1 \prec e_2$*,* $e_1 \prec e_3$*,* $e_2 \prec e_4$*,* $e_3 \prec e_4$*,* $e_2 \prec e_5$*, and* $e_3 \prec e_5$*. Timing constraints use one clock* $C = \{c_1\}$*, reset at* e_1 *(*$R(e_1) = \{c_1\}$*), with guards* $G(e_2) = (c_1 \leq 15)$ *and* $G(e_4) = (c_1 \leq 60)$*. This TPO is shown as the black DAG with black clock and guard annotations in Fig. 1.*

A *timed trace* $\tau = (\sigma_1, t^{(1)}) \ldots (\sigma_n, t^{(n)})$ is a sequence of event-timestamp pairs, where each $\sigma_i \in \Pi$ is distinct and $t^{(i)} \in \mathbb{R}_{\geq 0}$. A valid execution of a TPO is a timed trace that respects $\prec$ and timing constraints. To reason about the latter, we need to track clock readings, which we do via a *valuation* function $\nu : C \to \mathbb{R}_{\geq 0}$. This is fomalized below.

Definition 2 (TPO Semantics [8]). *A* run *of TPO* ϕ *is a sequence* $\rho = (\sigma_1, t^{(1)}, \nu_1) \ldots (\sigma_n, t^{(n)}, \nu_n)$*, where each* $\sigma_i \in \Pi$ *is distinct and each* ν_j *is a clock valuation, satisfying:*

- *timestamps are non-decreasing,* $t^{(1)} \leq \cdots \leq t^{(n)}$*;*
- $\sigma_1, \ldots, \sigma_n$ *is a linearization of* $\prec$*;*
- *just before* σ_j *fires (is executed), time duration* $\Delta t^{(j)} = t^{(j)} - t^{(j-1)}$ *elapses, advancing each clock* $c \in C$ *by* $\Delta t^{(j)}$*. Then, the resulting valuation* $\nu_{j-1} \oplus \Delta t^{(j)}$*, where notation* $(\nu \oplus \Delta t)(c) = \nu(c) + \Delta t$*, must satisfy the guard* $G(\sigma_j)$*; and*
- *after firing, the clocks in* $R(\sigma_j)$ *are reset to zero while all others are unchanged, giving* $\nu_j = reset(\nu_{j-1} \oplus \Delta t^{(j)}, R(\sigma_j))$*, where* $reset(\nu, \hat{C})(c) = 0$ *if* $c \in \hat{C}$*, else* $\nu(c)$*.*

A timed trace $\tau = (\sigma_1, t^{(1)}) \ldots (\sigma_n, t^{(n)})$ *is* compatible *with* ϕ *denoted* $\tau \models \phi$ *if there exists a corresponding run.*

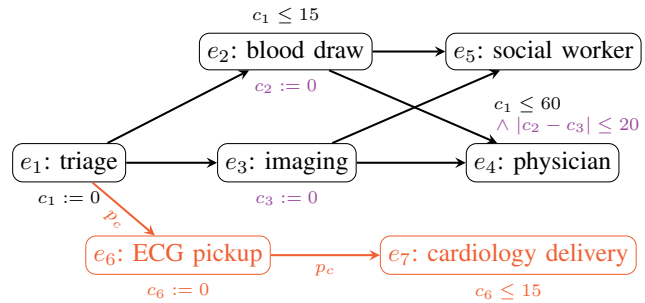


Fig. 1: TPO and cTPO specifications for the Hospital Robot examples. The TPO, shown with black edges and guards, captures the (original) requirements in Example 1. It is extended to a cTPO by adding the purple clocks and guards to represent the additional requirement R1, and the bottom orange branch to capture requirement R2.

An example for a compatible timed trace for the TPO in Figure 1 (Example 1) is $\tau_1 = (e_1, 0)(e_2, 10)(e_3, 20)(e_4, 30)(e_5, 40)$.

While expressive and interpretable, TPOs in Definition 1 have two major limitations: restricted guard conditions and the absence of environmental conditions.

B. Conditional TPOs with Complex Guards

Existing TPOs in Definition 1 only allow guard conditions with single-clock bounds (see G_C in Definition 1) and cannot express events triggered by external environmental conditions. For instance, in Example 1, additional requirements such as

R1: "*Blood draw* e_2 *and imaging* e_3 *must be done within 20 minutes of each other.*"

R2: "*If the robots happens to pass near the Cardiac Wing, it should also pick up ECG report* (e_6) *and deliver it to cardiologist* (e_7) *within 15 minutes.*"

cannot be expressed. We extend TPOs to support such expressions, enriching the properties they can represent while maintaining their strong interpretability. These extensions, however, have consequences in computation, which we elaborate in Section **??** and address in Section **??**.

First, we generalize the guard condition $\mathcal{G}_C$ to allow richer, *precedence-free* constraints. Specifically, each guard may

also be a conjunction of sums of clock differences, i.e.,

$$\mathcal{G}'_C = \mathcal{G}_C \cup \Big\{ \bigwedge_{k=1}^{N} \sum_{i_k \neq j_k \in I_k} (c_{i_k} - c_{j_k}) \bowtie t_k \mid N \leq 2^{|C|}, \\ I_k \subseteq \{1, \ldots, |C|\},\, t_k \in \mathbb{R}_{\geq 0} \Big\}. \quad (1)$$

This enables timing constraints between arbitrary pairs of events, regardless of their precedence relation (yielding *non-race-free* TPOs). Note that $\mathcal{G}'_C$ allows for conditions such as $|c_2 - c_3| \leq 20$, which precisely captures the additional requirement R1 for Example 1 by extending clocks to $C = \{c_1, c_2, c_3\}$, as shown in Figure 1 (purple clocks and guards).

The second extension is motivated by the fact that real robotic tasks rarely unfold as a single fixed task structure. Environmental features encountered along the way (i.e., a geologically significant outcrop, an obstacle requiring detour, a region triggering a safety protocol) may impose additional tasks with their own precedence and timing requirements, e.g., R2 for Example 1. The TPO formalism in Definition 1 cannot represent this: its event set, partial order, and timing constraints are fixed at specification time and do not adapt to runtime observations.

To capture such behavior, we associate events with logical conditions over the environment. Let $AP = \{p_1, \ldots, p_r\}$ be a finite set of atomic propositions (predicates), where each p_i corresponds to a region $\mathcal{R}_i$ of the environment and evaluates to true when the robot's trajectory passes through $\mathcal{R}_i$. Furthermore, let $\mathcal{F}(AP)$ denote the set of all Boolean (propositional) formulas over AP formed using $\neg$ (negation), $\wedge$ (conjunction), and $\vee$ (disjunction). We allow each event in Π to be triggered by a formula in $\mathcal{F}(AP)$. Intuitively, an event and its timing and precedence constraints are triggered in the task structure of TPO only when its associated formula evaluates to true. Otherwise, the event and its precedence and temporal constraints are ignored. We formalize this extension as a *Conditional TPO* (cTPO).

Definition 3 (Conditional TPOs)**.** *A* Conditional TPO *(cTPO) is a tuple $\phi_c = (\phi, AP, \alpha)$ where ϕ is a TPO as in Definition 1 with guard condition $\mathcal{G}'_C$ as in* (1)*, AP is a finite set of atomic propositions, and $\alpha : \Pi \to \mathcal{F}(AP)$ is an activation function that assigns, to each event $e \in \Pi$, a propositional condition (Boolean formula) $\alpha(e) \in \mathcal{F}(AP)$ over AP.*

When $\alpha(e)$ evaluates to true ($\alpha(e) \equiv \top$), event e and its associated precedence constraints, guard conditions, and clock resets are enforced; otherwise, e is not required. Note that cTPO ϕ_c recovers TPO ϕ by setting $\alpha(e) = \top$ for all $e \in \Pi$.

The TPO in Figure 1 with the bottom orange branch is a cTPO for Example 1 with R1 and R2 requirements, where $\Pi = \{e_1, \ldots, e_7\}$, $AP = \{p_c\}$, and $\alpha(e_i) = p_c$ if $i \in \{6, 7\}$ and $\top$ otherwise (not shown in the figure).

For each evaluation of the propositions in AP, a cTPO induces a TPO obtained by restricting to the events whose activation conditions evaluate to true. Hence, cTPO semantics are inherited from Definition 2 on this induced substructure.

Definition 4 (cTPO Semantics)**.** *Let $\omega : AP \to \{\bot, \top\}$ be a valuation of atomic propositions in AP. The* active event set *induced by ω is $\Pi_\omega := \{e \in \Pi \mid \alpha(e) \equiv \top$ under $\omega\}$, and* active suborder *induced by ω is $\prec_\omega := \prec \cap (\Pi_\omega \times \Pi_\omega)$. A* run *of a cTPO ϕ_c under ω is a sequence $\rho = (\sigma_1, t^{(1)}, \nu_1) \ldots (\sigma_k, t^{(k)}, \nu_k)$, where $k = |\Pi_\omega|$ and $\sigma_i \in \Pi_\omega$, satisfying the conditions in Definition 2 with $\prec$ replaced by $\prec_\omega$. A timed trace τ is* compatible *with ϕ_c under ω denoted $\tau \models \phi_c$ if there exists a corresponding run.*

An example of a compatible timed trace that satisfies the cTPO in Figure 1 (Example 1) is $\tau_1 = (e_1, 0)(e_2, 10)(e_3, 20)(e_4, 30)(e_5, 40)(e_6, 50)(e_7, 60)$.

C. Robot Model

Similar to [8], we consider a robot whose evolution in the environment is modeled as a Deterministic Transition System (DTS), which is an abstraction commonly made in the literature [17]–[19].

Definition 5 (DTS)**.** *A deterministic transition system (DTS) is a tuple [SE: $T = (X, x_0, A, \delta, \Delta, \Pi, L, AP, \ell)$], where*

- *X is a finite set of states,*
- *$x_0 \in X$ is the initial state,*
- *A is a finite set of actions,*
- *$\delta : X \times A \to X$ is the (partial) transition function,*
- *$\Delta : X \times A \to \mathbb{R}_{\geq 0}$ is the transition duration function,*
- *Π is a finite set of events, [SE:*
- *$L : X \to \Pi \cup \{\emptyset\}$ is an event labeling function with $L(x_0) = \emptyset$ that maps each state to an event,*
- *$AP = \{p_1, \ldots, p_r\}$ is a finite set of atomic propositions, and*
- *$\ell : X \to 2^{AP}$ is a proposition labeling function that maps each state to the atomic propositions it satisfies.]*

The region *associated with $p_k \in AP$ is $\mathcal{R}_k = \{x \in X \mid p_k \in \ell(x)\}$.*

A *plan* $\gamma = \gamma_0 \gamma_1 \ldots \gamma_{n-1}$ with $\gamma_i \in A$ is a sequence of actions. A plan γ is *valid* if it generates a trajectory $s^\gamma = s_0^\gamma s_1^\gamma \ldots s_n^\gamma$ such that $s_0^\gamma = x_0$ and $s_{i+1}^\gamma = \delta(s_i^\gamma, \gamma_i)$. The set of all valid plans is denoted Γ. The *duration* of s^γ is defined as the sum of the transition durations along the trajectory, i.e., $D(s^\gamma) = \sum_{i=0}^{n-1} \Delta(s_i^\gamma, \gamma_i)$.

A valid plan γ induces a timed trace $\tau^\gamma = (L(s_0^\gamma), t_0) \ldots (L(s_n^\gamma), t_n)$, where $t_i = D(s_0^\gamma, \ldots, s_i^\gamma)$. We say plan γ *satisfies* a TPO ϕ defined over events Π if $\tau^\gamma \models \phi$ per Definition 2. For a cTPO $\phi_c = (\phi, AP, \alpha)$, we first define valuation ω for every $p_k \in AP$ to be

$$\omega(p_k) := \begin{cases} \top & \text{if } s_i^\gamma \in \mathcal{R}_k \text{ for some } i \\ \bot & \text{otherwise.} \end{cases} \quad (2)$$

Then, γ satisfies ϕ_c if τ^γ is compatible with ϕ_c under ω per Definition 4.

Our goal is to compute a minimum-duration plan that satisfies a given cTPO specification, as formalized below.

Problem 1 (Plan Synthesis)**.** *Given robot as a DTS T in Definition 5 and a cTPO task ϕ_c as in Definition 3, synthesize a valid plan $\gamma^* \in \Gamma$ that generates a trajectory with minimum duration and satisfies ϕ_c, i.e.,*

$$\gamma^* = \arg\min_{\gamma \in \Gamma} D(s^\gamma) \quad \textit{subject to} \quad \tau^\gamma \models \phi_c.$$

III. Approach

Our solution to Problem 1 proceeds in three stages. First, we abstract the DTS T and cTPO ϕ_c into a weighted directed graph and show that cTPO compatibility reduces to a system of linear difference constraints over event timestamps, casting the problem as an instance of the Generalized Traveling Salesman Problem with Time Windows and Precedence Relations (GTSP-TWPR) [2], [8] (Section III-A). Second, we solve the GTSP-TWPR instance exactly via an augmented Mixed-Integer Linear Program (MILP). We extend the formulation of [8] by introducing atomic-proposition tracking and formula evaluation constraints; these ensure that event activation $\alpha(e)$ and its associated timing requirements are only enforced when their specific conditions are satisfied (Section III-B). Third, to reduce the computational cost as problem size grows, we introduce a sub-TPO decomposition that identifies self-contained unconditional substructures, solves each independently as a smaller MILP, and composes the results into a globally optimal plan (Section **??**).

A. Reduction to GTSP-TWPR

The key observation underlying the reduction is that cTPO compatibility can be expressed purely in terms of event timestamps, without reference to clock valuations.

Proposition 1 (Timestamp Characterization [2])**.** *Let $\omega : AP \to \{\bot, \top\}$ be a fixed valuation. A timed trace $\tau = (\sigma_1, t^{(1)}) \cdots (\sigma_k, t^{(k)})$ over Π_ω is compatible with ϕ_c under ω if and only if*

$$t^{(i)} \in [a_i,\, b_i], \tag{3a}$$

$$t^{(j)} - t^{(i)} \in [\ell_{ij},\, u_{ij}], \tag{3b}$$

$$t^{(l)} - t^{(k)} \in [f_{kl},\, w_{kl}], \tag{3c}$$

for all $e_i \in \Pi_\omega$ in (3a)*, all ordered pairs $e_i \prec e_j$ in Π_ω in* (3b)*, and in* (3c) *for all pairs $e_k, e_l \in \Pi_\omega$ such that a clock-difference conjunct $(c_k - c_l) \bowtie a$ appears in G, where c_k and c_l are reset at e_k and e_l respectively. All bounds are derived from G and R.*

Proposition 1 recasts cTPO compatibility as a system of linear difference constraints. Condition (3a) imposes an *absolute time window* on each event, derived from guards whose clocks are reset at mission start. Condition (3b) imposes a *relative time window* between each pair of precedence-ordered events, derived from guards whose clocks are reset at the earlier event. Condition (3c) imposes a *precedence-free relative time window* between events coupled by clock-difference conjuncts in G. These three constraint classes map one-to-one onto the time windows and precedence constraints of the GTSP-TWPR.

Definition 6 (GTSP-TWPR [8])**.** *Let $G = (V, E)$ be a weighted directed graph with depot $v_0 \in V$. For each edge $(v_i, v_j) \in E$, let $d_{ij} \in \mathbb{R}_{\geq 0}$ denote its* travel cost*. Nodes in $V \setminus \{v_0\}$ are partitioned into two kinds:* event nodes*, grouped into n disjoint* event groups *$V_1, \ldots, V_n$ with dwell cost $d_i \in \mathbb{R}_{\geq 0}$; and* waypoint nodes*, which belong to no event group and have $d_i = 0$. Let $\tau_i \in \mathbb{R}_{\geq 0}$ denote the visit time of the selected node in group V_l. A subset of event groups may be designated* inactive *and skipped entirely. The tour minimizes makespan, starts and ends at v_0, visits exactly one node from each non-inactive event group, and is subject to:*

1) time windows: *$\tau_i \in [a_i, b_i]$ with $a_i, b_i \in \mathbb{R}_{\geq 0}$ and for each group V_i;*
2) precedence constraints: *$\tau_i \leq \tau_j$ for each ordered pair of groups $V_i \prec V_j$; and*
3) relative time windows: *$\tau_j - \tau_i \in [\ell_{ij}, u_{ij}]$ with $l_{ij}, u_{ij} \in \mathbb{R}$ for selected pairs of groups (V_i, V_j).*

a) Translation of Problem 1 to GTSP-TWPR: By Proposition 1, cTPO compatibility under a valuation ω reduces exactly to constraints of the form (3a)–(3c), which map onto conditions (1)–(3) of Definition 6: constraint (3a) yields the time windows (a); constraint (3b) yields the precedence-ordered relative windows (b)–(c) for pairs $e_i \prec e_j$; and constraint (3c) yields additional relative windows (c) for pairs coupled by clock-difference conjuncts in G. Event groups corresponding to conditional events e_l with $\alpha(e_l) \neq \top$ are designated inactive when $\alpha(e_l)$ evaluates to false under the induced valuation ω.

We construct $G = (V, E)$ from T and ϕ_c as follows.

b) Node set: Since nodes of G correspond directly to states of T, we identify $v_i \in V$ with a state $x_i \in X$ and write:

$$V = \{x \in X \mid x = x_0 \text{ or } L(x) \neq \emptyset \text{ or } \ell(x) \neq \emptyset\}. \tag{3}$$

The depot is $v_0 = x_0$. Event groups are $V_l = \{x \in X \mid L(x) = e_l\}$ for each $e_l \in \Pi$. States with $\ell(x) \neq \emptyset$ but $L(x) = \emptyset$ form the *waypoint nodes* $V_\ell = \{x \in X \mid L(x) = \emptyset,\, \ell(x) \neq \emptyset\}$; these nodes are never selected as event representatives but may be visited as intermediate stops for proposition tracking (see Section III-B). A state with both $L(x) \neq \emptyset$ and $\ell(x) \neq \emptyset$ belongs to its event group and contributes to proposition tracking simultaneously.

c) Edges and costs.: A directed edge $(v_i, v_j) \in E$ is added for each pair in Table I. Edge costs d_{ij} are shortest-

v_i	v_j	Condition
$v_i \in V_l$	$v_j \in V_{l'}$	$e_l \prec e_{l'}$
$v_i \in V_l$	$v_j \in V_{l'}$	$e_l \not\prec e_{l'}$
$v_i \in V_l$	$v_j \in V_{l'}$	$t^{(l')} - t^{(l)} \in [\ell_{ll'}, u_{ll'}]$
$v_i \in V_{wp} \cup \{v_0\}$	$v_j \in V_l$,	$\forall\, l \in [n]$
$v_i \in V_l$	$v_j \in V_{wp} \cup \{v_0\}$	$\forall\, l \in [n]$
$v_i \in V_{wp}$	$v_j \in V_{wp}$	

TABLE I: Edge set construction.

path durations computed via Floyd–Warshall over all of X

$$\min \tau_{N+1}$$

$$\sum_i y_{ij} - \sum_k y_{jk} = 0, \qquad j \in I \cup [N] \tag{4a}$$

$$\sum_{x_j \in V_l} \sum_i y_{ij} = 1, \qquad l \in [n],\ \alpha(e_l) = \top \tag{4b}$$

$$\sum_{x_j \in V_l} \sum_k y_{jk} = 1, \qquad l \in [n],\ \alpha(e_l) = \top \tag{4c}$$

$$\tau_j - \tau_i \bowtie a_{ij}, \qquad (e_i, e_j) \in \text{(3b)} \tag{4d}$$

$$\tau_l - \tau_k \bowtie b_{kl}, \qquad (e_k, e_l) \in \text{(3c)} \tag{4e}$$

$$y_{ij} = 1 \Rightarrow \tau_j - \tau_i \geq d_{ij} + d_j, \qquad i \in I \cup [N],\ j \in [N] \tag{4f}$$

$$y_{ij} = 1 \Rightarrow \tau_{N+1} - \tau_i \geq d_{ij}, \qquad i \in [N],\ j \in I \tag{4g}$$

$$y_{ij} \in \{0,1\}, \quad \tau_i \geq 0 \tag{4h}$$

$$z_k \geq y_{ij}, \qquad (i,j) \in E_k \tag{4i}$$

$$z_k \leq \sum_{(i,j) \in E_k} y_{ij}, \qquad \forall\, k \tag{4j}$$

$$\sum_{x_j \in V_l} \sum_i y_{ij} = z_e, \qquad l \in [n],\ \alpha(e_l) \neq \top \tag{4k}$$

$$z_e = 1 \Rightarrow \tau_j - \tau_i \in [\ell_{ij},\, u_{ij}], \qquad (e_i, e_j) \in \text{(3b)} \tag{4l}$$

$$z_e = 1 \Rightarrow \tau_l - \tau_k \in [f_{kl},\, w_{kl}], \qquad (e_k, e_l) \in \text{(3c)} \tag{4m}$$

$$z_k \in \{0,1\}, \quad z_e \in \{0,1\} \tag{4n}$$

Fig. 2: MILP formulation. $N{=}|V|$, $[N]{=}\{1,\ldots,N\}$, $I{=}\{0\}$, e_l = event of group V_l, $P_k{=}\{i \in I{\cup}[N] \mid p_k \in \ell(x_i)\}$, $E_k{=}\{(i,j) \in E \mid i \in P_k \text{ or } j \in P_k\}$. Constraints (4a)–(4h) follow [8]; (4i)–(4n) are novel.

and read off for pairs in V. The dwell cost is $d_i = \Delta(x_i, a)$ if v_i has a self-loop action $a \in A$, else $d_i = 0$.

B. MILP Formulation

We solve the GTSP-TWPR instance exactly using the MILP in Figure 2. Constraints (4a)–(4h) follow [8] for the unconditional TPO case. The novel contributions of this work are constraints (4i)–(4n), which extend the formulation to handle the conditional structure of cTPOs: the *proposition tracking* constraints (4i)–(4j) determine which atomic propositions evaluate to true, the *formula evaluation* constraints (**??**) derive per-event activation variables, the *conditional coverage* constraint (4k) ties tour structure to activation, and the *conditional timing* constraints (4l)–(4m) activate the associated timing requirements only when the corresponding activation condition is satisfied.

For each conditional event e_l with $\alpha(e_l) \neq \top$, the activation variable $z_e \in \{0,1\}$ is defined by linearizing $\alpha(e_l)$ over the proposition variables $\{z_k\}$:

$$\begin{aligned}
\alpha(e_l) = p_k &: \quad z_e = z_k, \\
\alpha(e_l) = \neg p_k &: \quad z_e = 1 - z_k, \\
\alpha(e_l) = \varphi_1 \wedge \varphi_2 &: \quad z_e \leq z_{\varphi_1},\ z_e \leq z_{\varphi_2},\ z_e \geq z_{\varphi_1} + z_{\varphi_2} - 1, \\
\alpha(e_l) = \varphi_1 \vee \varphi_2 &: \quad z_e \geq z_{\varphi_1},\ z_e \geq z_{\varphi_2},\ z_e \leq z_{\varphi_1} + z_{\varphi_2}.
\end{aligned}$$

General formulas are handled recursively.

IV. Experiments

We evaluate the approach on two case studies demonstrating cTPO plan synthesis, followed by benchmarks assessing the computational benefit of the sub-TPO decomposition. The case studies illustrate how the conditional structure of a cTPO produces qualitatively different plans depending on whether a proposition evaluates to true or false under the executed trajectory. The benchmarks then compare the decomposition formulation against the monolithic formulation on randomly generated TPOs.

A. Conditional TPO Plans

1) Mars Rover Atmospheric Science Mission: We consider a Mars rover mission in which the rover must visit two science sites to perform measurements and return to its lander. If a geologically significant outcrop is encountered, the rover must execute an additional time-bounded sampling sequence. The resulting cTPO is shown in Fig. 3c.

We evaluate this specification in two environments differing only in the placement of $\mathcal{R}_{\text{o}}$, the outcrop region, relative to the lander (Figs. 3a, 3b). In the first, the optimal trajectory passes through $\mathcal{R}_{\text{o}}$, setting $p_{\text{o}} = \top$ and obliging execution of e_6 and e_7; in the second it does not, yielding $p_{\text{o}} = \bot$ and a plan over the five unconditional events only.

2) Office Cleaning Robot: We consider a cleaning robot tasked with completing a route and returning to its charging station within a fixed time bound. If the robot traverses a puddle region $\mathcal{R}_{\text{p}}$, it must drive over a carpet to dry off before charging. The resulting cTPO is shown in Fig. 4c.

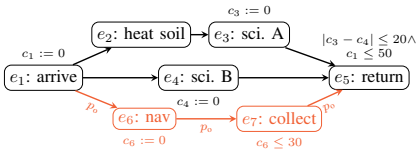
We evaluate this specification on two robot models with different traversal costs in the same environment. A water-resistant robot incurs no penalty for crossing $\mathcal{R}_{\text{p}}$, so its optimal trajectory passes through it, setting $p_{\text{p}} = \top$ and including the drying sequence (e_3, e_4). A non-water-resistant robot incurs a high cost, so its optimal trajectory avoids $\mathcal{R}_{\text{p}}$, yielding $p_{\text{p}} = \bot$ and a plan over the two unconditional events only (Figs. 4a, 4b).

B. Benchmarks

All benchmarks were executed on a MacBook Air M3 with 16GB RAM. The benchmark construction follows a hierarchical sequence to evaluate the computational benefits of sub-TPO decomposition. First, a high-level TPO is sampled as a random DAG using $n = 30$ events. The structural density is controlled by $P_{\text{hl}} = 0.6$, the probability of a precedence edge between any two events, while the temporal density is set by $P_{\text{local}} = 0.5$, the probability that an edge is constrained by a local timing window. We select $k = 5$ events from this

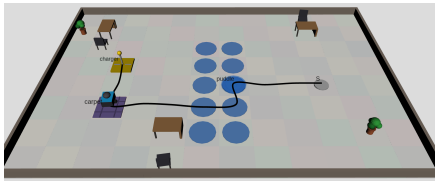
(a) $\mathcal{R}_o$ surrounds the lander. The optimal trajectory passes through $\mathcal{R}_o$, activating $p_o = \top$ and extending the plan with e_6 and e_7.

(b) $\mathcal{R}_o$ is placed away from the lander. The optimal trajectory avoids $\mathcal{R}_o$, yielding $p_o = \bot$ and a plan over five events only.

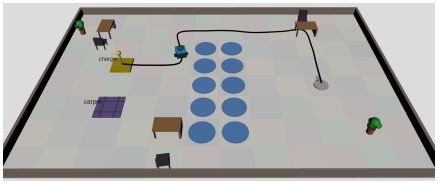


(c) TPO for the Mars Rover Atmospheric Science Mission. Black edges and guards apply unconditionally. Red-orange denotes the conditional events, edges, clocks and guards.

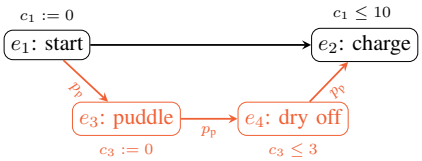
Fig. 3: Synthesized plans and the corresponding conditional task plan operator (cTPO) for the Mars Rover performing an atmospheric science mission. Subfigures illustrate trajectory differences and conditional events based on whether the rover traverses the outcrop region $\mathcal{R}_o$.



(a) Water-resistant robot: optimal trajectory passes through $\mathcal{R}_p$, activating $p_p = \top$.



(b) Non-water-resistant robot: optimal trajectory avoids $\mathcal{R}_p$, yielding $p_p = \bot$.



(c) cTPO for the Office Cleaning Robot. Black edges/guards are unconditional; red-orange denotes conditional events activated by p_p.

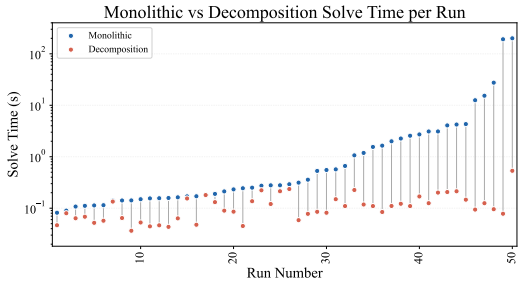
Fig. 4: Synthesized plans and the corresponding conditional task plan operator (cTPO) for two robot models navigating the same office environment. Subfigures illustrate trajectory differences and conditional events based on water-resistance.

graph to serve as insertion points – placeholders designated for replacement by sub-TPOs.

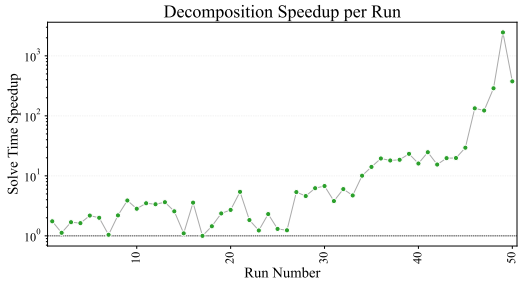
For each insertion point, a sub-TPO (5–7 events) is generated as a random DAG with internal edge probability $P_{\text{edge}} = 0.3$. Internal timing constraints are added with probability P_{local}. Each sub-TPO is modified with synthetic entry/exit events to satisfy the SESE property and replaces a placeholder event by redirecting incident edges to these new points, ensuring isolation per Definition **??**. This process results in a final TPO of approximately 55–65 events.

The resulting TPOs were evaluated across 50 feasible runs, sorted by increasing monolithic solve time, where each run's solve time is averaged over 3 trials. Figure 5a compares absolute solve times, showing that the decomposition method consistently outperforms the monolithic solver across all runs. While monolithic solve times grow sharply with problem complexity–exceeding 100 seconds for the hardest instances–decomposition times remain consistently below 1 second. Figure 5b shows the resulting speedup ratio, defined as the ratio of the average monolithic solve time to the average decomposition solve time. The speedup increases with problem difficulty and exceeds three orders of magnitude for the most complex instances.

The runs in which monolithic solve time increases most sharply correspond to sparser timing constraints, which provide less pruning of the search space and confirm the dual role of timing constraints discussed in Section **??**.



(a) Absolute solve times (blue: monolithic, red: decomposition)



(b) Speedup ratio (T_{mono}/T_{decom}).

Fig. 5: Performance comparison of monolithic and decomposition-based planning across feasible runs, sorted by monolithic solve time.

V. Conclusion

In this work, we introduced Conditional Timed Partial Orders (CTPOs) to address the expressiveness limitations of existing TPOs in robotic task planning planning. By incorporating generalized clock-difference guards and environment-dependent activation conditions, we enabled the specification of complex, precedence-free timing constraints alongside reactive, conditionally activated task structures. To efficiently solve the resulting plan synthesis problem, we extended

existing Mixed-Integer Linear Programming (MILP) formulations and introduced a novel sub-TPO decomposition algorithm. Our benchmarks and case studies demonstrate that this decomposition preserves the global optimality of the synthesized plans, yields computational speedups exceeding three orders of magnitude for complex instances, and exposes the modular structure of missions to significantly enhance designer interpretability.

REFERENCES

[1] H. Guo, F. Wu, Y. Qin, R. Li, K. Li, and K. Li, "Recent trends in task and motion planning for robotics: A survey," *ACM Computing Surveys*, 2023.

[2] K. Watanabe, G. Fainekos, B. Hoxha, M. Lahijanian, D. Prokhorov, S. Sankaranarayanan, and T. Yamaguchi, "Timed partial order inference algorithm," in *Int. Conference on Automated Planning and Scheduling*, vol. 33, no. 1, 2023, pp. 639–647.

[3] R. Alur and D. L. Dill, "A theory of timed automata," *Theoretical computer science*, vol. 126, no. 2, pp. 183–235, 1994.

[4] E. Asarin, O. Maler, A. Pnueli, and J. Sifakis, "Controller synthesis for timed automata," *IFAC Proceedings Volumes*, vol. 31, no. 18, pp. 447–452, 1998.

[5] C. E. Noon, *The generalized traveling salesman problem*. University of Michigan, 1988.

[6] A. Mingozzi, L. Bianco, and S. Ricciardelli, "Dynamic programming strategies for the traveling salesman problem with time window and precedence constraints," *Operations research*, vol. 45, no. 3, pp. 365–377, 1997.

[7] P. C. Pop, O. Cosma, C. Sabo, and C. P. Sitar, "A comprehensive survey on the generalized traveling salesman problem," *European Journal of Operational Research*, 2023.

[8] K. Watanabe, G. Fainekos, B. Hoxha, M. Lahijanian, H. Okamoto, and S. Sankaranarayanan, "Optimal planning for timed partial order specifications," in *Int. Conf. on Robotics and Auto.* IEEE, May 2024.

[9] A. Schrijver, *Theory of Linear and Integer Programming*, ser. Wiley Series in Discrete Mathematics and Optimization. John Wiley & Sons, 1998.

[10] P. Vansteenwegen, W. Souffriau, and D. Van Oudheusden, "The traveling salesman problem with time windows," in *Vehicle Routing: Problems, Methods, and Applications*, P. Toth and D. Vigo, Eds. Philadelphia, PA, USA: Society for Industrial and Applied Mathematics, 2011, pp. 121–130.

[11] M. R. Garey and D. S. Johnson, *Computers and Intractability: A guide to the theory of NP-Completeness*. W.H.Freeman, 1979.

[12] R. Koymans, "Specifying real-time properties with metric temporal logic," *Real-Time Systems*, vol. 2, no. 4, pp. 255–299, 1990.

[13] U. A. Fiaz and J. S. Baras, "A hybrid compositional approach to optimal mission planning for multi-rotor uavs using metric temporal logic," in *IFAC-PapersOnLine*, vol. 53, no. 2, 2020, pp. 15 404–15 411.

[14] Y. Zhou, D. Maity, and J. S. Baras, "Optimal mission planner with timed temporal logic constraints," in *European Control Conference*, 2015.

[15] R. Dechter, I. Meiri, and J. Pearl, "Temporal constraint networks," *Artificial intelligence*, vol. 49, no. 1-3, pp. 61–95, 1991.

[16] H. Kress-Gazit, G. E. Fainekos, and G. J. Pappas, "Temporal-logic-based reactive mission and motion planning," *IEEE Transactions on Robotics*, vol. 25, no. 6, pp. 1370–1381, 2009.

[17] M. L. Hadas Kress-Gazit and V. Raman, "Synthesis for robots: Guarantees and feedback for robot behavior," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 1, 2018.

[18] M. Lahijanian, M. Kloetzer, S. Itani, C. Belta, and S. B. Andersson, "Automatic deployment of autonomous cars in a robotic urban-like environment (rule)," in *Int Conf on Robotics and Auto*, 2009.

[19] H. Kress-Gazit, G. E. Fainekos, and G. J. Pappas, "Where's waldo? sensor-based temporal logic motion planning," in *Int. Conf. on Robotics and Auto.* IEEE, 2007, pp. 3116–3121.